



# Linux ACL (Access Control Lists) — Complete Practical Notes

## 1. Why ACLs exist (core idea)

Traditional Linux permissions allow **only 3 classes**:

- owner
- group
- others

ACLs extend this to allow:

- **multiple users**
- **multiple groups**
- **fine-grained control**

👉 ACLs solve the *"I want user A and group B to access this, but not everyone else"* problem.

## 2. When ACLs are needed (real use cases)

- Shared project directories
- Multi-user servers
- CI/CD workspaces
- Docker volumes
- NFS / shared mounts
- Giving temporary access without changing ownership

If you're doing **team work**, ACLs are the right tool.

## 3. Basic ACL concepts

Each file/directory can have:

### 1. Access ACL

→ applies to existing files

### 2. Default ACL

→ inherited by newly created files/directories

→ only meaningful on directories

## 4. Prerequisites

### Check ACL support

```
setfacl --version
```

Most modern Linux systems support ACLs by default.

## 5. View ACLs — getfacl

```
getfacl file_or_dir
```

Example output:

```
# file: project
# owner: alice
```

```
# group: developers
user::rwx
user:bob:rw-
group::r-x
mask::rw-
other::---
```

### Meaning:

- user:: → owner permissions
- user:bob → ACL for user bob
- group:: → owning group
- mask:: → **maximum effective permission**
- other:: → others

## 6. Add or modify ACL — setfacl -m

### User ACL

```
setfacl -m u:alice:rw file.txt
```

### Group ACL

```
setfacl -m g:developers:rwx dir/
```

### Multiple entries at once

```
setfacl -m u:alice:rw,g:qa:r file.txt
```

## 7. Recursive ACLs (existing files)

```
setfacl -R -m u:alice:rwX project/
```

### Why X (capital X)?

- Applies x only to:
  - directories
  - already executable files
- Prevents breaking normal files

## 8. Default ACLs (inheritance) ★ MOST IMPORTANT

### Set default ACL

```
setfacl -d -m u:alice:rwX project/
```

### Recursive default ACL

```
setfacl -R -d -m g:developers:rwX project/
```

### Behavior:

- New files inherit ACLs
- New directories inherit **default ACLs**

👉 **ACLs do NOT retroactively apply unless -R is used**

## 9. The ACL Mask (VERY IMPORTANT)

### What is the mask?

The mask limits the **maximum effective permissions** for:

- named users
- named groups

Example:

user:alice:rwX

mask::r--

Effective permission = r-- ❌

Even though rwX is set.

### Fix mask automatically

```
setfacl -m u:alice:rwX file.txt
```

(setfacl recalculates mask by default)

### Fix mask manually

```
setfacl -m m:rwX file.txt
```

## 10. How ACLs affect ls -l

If you see:

-rw-rwx---+

The + means:

**This file has ACLs**

Use getfacl to see details.

## 11. ACLs vs Deletion (CRITICAL RULE)

ACLs **do NOT** change Linux deletion rules.

To delete a file:

- user must have w + x on the **parent directory**

**File ACL alone** ❌

```
setfacl -m u:alice:rw file.txt
```

→ cannot delete

**Directory ACL** ✅

```
setfacl -m u:alice:rwX mydir
```

→ can delete files inside

## 12. Sticky bit interaction

If directory has sticky bit:

drwxrwxrwt

Then even with ACL:

- only file owner
- directory owner
- root

can delete files

ACL **cannot** override sticky bit.

## 13. Removing ACLs (cleanup)

**Remove all ACLs**

```
setfacl -b file_or_dir
```

**Remove only default ACLs**

```
setfacl -k directory/
```

## 14. Copying files & ACLs

Preserve ACLs:

```
cp -a source dest
```

```
rsync -A source dest
```

Without -a / -A, ACLs are lost.

## 15. Best-practice shared directory setup (REAL WORLD)

```
chgrp developers project/
```

```
chmod 2775 project/
```

```
setfacl -R -m g:developers:rwX project/
```

```
setfacl -R -d -m g:developers:rwX project/
```

This ensures:

- correct group
- group inheritance
- permission inheritance
- no permission drift

## 16. Common mistakes ❌

❌ Forgetting permissions in -m

```
setfacl -m g:dev project ❌
```

✅ Correct:

```
setfacl -m g:dev:rwX project
```

❌ Forgetting default ACLs

❌ Ignoring the mask

❌ Using x instead of X

## 17. ACL vs chmod vs chown (mental map)

| Tool       | Purpose             |
|------------|---------------------|
| chmod      | basic permissions   |
| chown      | ownership           |
| chgrp      | group               |
| ACL        | fine-grained access |
| setgid     | group inheritance   |
| sticky bit | delete protection   |